

Claude Code — Models Guide

Which Model to Use & When to Switch

Source: Official Claude Code Docs (code.claude.com/docs/en/model-config)

Last updated: April 2026 | Claude Code v2.1.x

Available Models at a Glance

Alias	Current Model	Context Window	Best For
sonnet	Claude Sonnet 4.6	200K (1M on Max/Team)	Daily coding — the default
opus / best	Claude Opus 4.6	1M (GA on Max/Team/Enterprise)	Complex reasoning, deep analysis
haiku	Claude Haiku 4.5	200K	Fast, simple, cheap tasks
sonnet[1m]	Sonnet 4.6 (1M context)	1M	Very long sessions, large codebases
opus[1m]	Opus 4.6 (1M context)	1M	Same but with Opus reasoning depth
opusplan	Opus → Sonnet hybrid	—	Plan with Opus, execute with Sonnet

Aliases always point to the latest version. To pin a specific version, use the full model name (e.g. `claude-opus-4-6`).

Full Model IDs (Pinned Versions)

Use these when you need version stability — enterprise deployments, reproducible workflows, or environment variable configuration.

```
claude-opus-4-6
claude-sonnet-4-6
claude-sonnet-4-5-20250929
claude-opus-4-5-20251101
claude-haiku-4-5-20251001
```

What Each Model Is Actually Like

● Sonnet 4.6 — Your Daily Driver

The default for a reason. Strong reasoning applied to everyday work — fast enough for real-time collaboration, capable enough that most problems won't outgrow it.

What it handles well:

- Writing new features and components

- Fixing standard bugs
- Refactoring code
- Writing tests
- Generating documentation
- PR creation and code reviews
- Computer use and vision tasks
- Document and spreadsheet creation

When Sonnet is the right choice: If you're not sure, start here. 90%+ of coding tasks don't require anything more.

● **Opus 4.6 — The Deep Thinker**

Exceptional for specialized complex tasks requiring advanced reasoning. Built for problems that genuinely need sustained, deep thinking. Uses more of your rate limit — reserve it for tasks that truly need it.

What it handles well:

- Hard bugs — especially intermittent, race conditions, concurrency issues
- Architecture and system design decisions
- Evaluating trade-offs between multiple approaches
- Security audits and vulnerability analysis
- Complex multi-file refactors with deep interdependencies
- Understanding unfamiliar or legacy codebases
- Multi-agent coordination (Agent Teams)
- Tasks requiring 1M context — entire repositories, massive codebases

When Opus is the right choice: When Sonnet has tried twice and keeps going off-track, or when you need sustained reasoning across a very long context.

● **Haiku 4.5 — The Sprinter**

Fast, lightweight, and efficient with your rate limit. Built for everyday quick requests. Rivals the reasoning capabilities of older Sonnet models at a fraction of the cost.

What it handles well:

- Bulk find-and-replace operations (updating copyright headers, renaming variables)
- Simple lookups ("what's the syntax for X?")
- Quick summaries and short answers
- File formatting and linting passes
- Scaffolding boilerplate
- Tasks where speed matters more than depth

When Haiku is the right choice: When the task is mechanical, repetitive, or doesn't require judgment. Using Opus on a task Haiku could handle wastes tokens and slows you down.

🔗 **opusplan — The Intelligent Hybrid**

A special mode built into Claude Code. Uses Opus during Plan Mode for deep reasoning and architecture decisions, then automatically switches to Sonnet for code generation and implementation.

What it handles well:

- Large feature implementations where you want a thorough plan but fast execution
- Complex migrations (e.g. OAuth2 migration, API version upgrades)

- Refactors that touch many files
- Any work where planning quality matters more than execution cost

How to use it:

```
claude --model opusplan
# or mid-session:
/model opusplan
```

Default Model by Plan

Plan	Default Model
Max & Team Premium	Opus 4.6
Pro & Team Standard	Sonnet 4.6
Enterprise	Opus 4.6 available but not default
API Key users	Sonnet 4.6

Claude Code may automatically fall back to Sonnet if you hit a usage threshold with Opus on certain plans.

How to Switch Models — All Methods

1. Mid-Session (Most Common)

```
/model sonnet      # switch to Sonnet
/model opus        # switch to Opus
/model haiku       # switch to Haiku
/model opusplan    # enable plan/execute hybrid
/model default     # revert to your plan's default
```

Inside `/model`, use **left/right arrow keys** to also adjust the effort level for the selected model.

2. At Startup (For a Specific Session)

```
claude --model opus
claude --model sonnet
claude --model haiku
claude --model opusplan
claude --model claude-opus-4-6    # pinned version
```

3. Environment Variable (Persistent Default)

```
export ANTHROPIC_MODEL="opus"    # or sonnet, haiku

# Add to ~/.zshrc or ~/.bashrc to make permanent
```

```
echo 'export ANTHROPIC_MODEL="sonnet"' >> ~/.zshrc
source ~/.zshrc
```

4. Settings File (Project or Global)

```
// .claude/settings.json (project-level)
// ~/.claude/settings.json (global)
{
  "model": "sonnet"
}
```

5. One-Shot / Headless Command

```
claude --model opus -p "Analyse this architecture and suggest improvements"
claude --model haiku -p "Update copyright year to 2026 in all files"
```

Effort Levels — Independent from Model

Effort level and model choice are **separate controls**. You can have Sonnet at max effort, or Opus at low effort.

Level	Behaviour	Persists Across Sessions
low	Fast, less thorough	Yes
medium	Default for Pro/Max	Yes
high	Deep reasoning	Yes
max	Maximum — Opus 4.6 only	No (current session only)
auto	Reset to model default	—

```
/effort low
/effort medium
/effort high
/effort max      # Opus 4.6 only
/effort auto     # reset to default
```

Or use the arrow keys inside `/model` to adjust effort alongside model selection.

Default effort by plan:

- Pro and Max subscribers → `medium` effort
- API key, Team, Enterprise, Bedrock/Vertex users → `high` effort

Effort + Model Combinations

Combination	Use For
-------------	---------

Haiku + low	Bulk simple tasks, maximum speed
Sonnet + medium	Default daily coding (most common)
Sonnet + high	Harder problems without paying for Opus
Opus + high	Architecture decisions, deep debugging
Opus + max	Your hardest problems — use sparingly

The `ultrathink` Keyword

Claude Code only — does not work in `claude.ai` chat or the API.

Adding `ultrathink` anywhere in a prompt triggers **high effort** for that single turn, then reverts to the session default.

```
ultrathink - why is this auth flow failing intermittently?

Analyse this microservice architecture and find failure points. ultrathink.
```

When to use `ultrathink` vs `/effort` :

Method	Scope	Best When
<code>ultrathink</code> in prompt	Single turn only	Mid-conversation, one hard problem
<code>/effort high</code>	Whole session	Want deep reasoning throughout
<code>/effort max</code>	Whole session (Opus only)	Hardest problems, entire session

Practical Model-Switching Workflow

A powerful pattern — switch models within a single session as task complexity changes:

```
> Scaffold the new payments module
[Sonnet handles this well - fast, reliable]

> /model opus
> The payment flow is failing on retry logic -
  ultrathink through all possible race conditions
[Opus for deep debugging]

> /model haiku
> Update the copyright year to 2026 in all file headers
[Haiku for bulk mechanical task]

> /model sonnet
> Write unit tests for the retry logic
[Back to Sonnet for standard dev work]
```

When to Switch — Decision Guide

Stay on Sonnet when:

- Writing new features or components
- Fixing standard bugs
- Refactoring code
- Writing tests or documentation
- Creating PRs and reviewing code
- You want fast output without burning through rate limits

Switch to Opus when:

- Debugging a hard, intermittent, or race-condition bug that Sonnet keeps missing
- Designing a new architecture or evaluating design patterns
- Analysing a large or unfamiliar codebase for the first time
- Conducting a security audit or vulnerability review
- You've tried Sonnet twice and it keeps going off-track
- You need 1M context for an entire repository

Switch to Haiku when:

- Updating boilerplate or copyright headers across many files
- Simple find-and-replace-style tasks
- Quick factual lookups
- File formatting or linting passes
- You're near a rate limit and need to conserve tokens

Switch to opusplan when:

- Building a large feature that needs a thorough plan before any code
- Running a complex migration across many files
- You want Opus intelligence during planning but Sonnet speed during execution

Switch to [1m] context variants when:

- Working with a monorepo or large codebase where Claude keeps losing context
- Sessions that span dozens of files and must retain everything
- Long-running agent tasks that chain many tool calls

Rule of Thumb

Using Opus on a task Haiku could handle wastes tokens for no gain and slows you down.

Start with Sonnet. Escalate to Opus only when you genuinely hit its ceiling. Drop to Haiku for anything mechanical or repetitive.

Environment Variables for Model Configuration

```
# Set default model
export ANTHROPIC_MODEL="sonnet"

# Pin specific versions (for enterprise/Bedrock/Vertex stability)
```

```
export ANTHROPIC_DEFAULT_OPUS_MODEL="claude-opus-4-6"
export ANTHROPIC_DEFAULT_SONNET_MODEL="claude-sonnet-4-6"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="claude-haiku-4-5-20251001"

# Set effort level persistently
export CLAUDE_CODE_EFFORT_LEVEL="high"

# Control thinking token budget (0 = disable thinking)
export MAX_THINKING_TOKENS=31999

# Disable adaptive thinking (revert to fixed budget)
export CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING=1
```

Checking Your Current Model

```
/status      # shows current model, effort level, account, version
/model       # opens model picker – highlights currently active model
```

The current effort level is also displayed next to the logo and spinner in the terminal (e.g. "with low effort") so you can confirm without opening `/model` .

Extended Context (1M Tokens)

- Available on **Max, Team, and Enterprise plans** (enabled by default as of v2.1.75)
- Use `sonnet[1m]` or `opus[1m]` aliases, or just `sonnet` / `opus` on supported plans
- Useful for entire repositories, large codebase analysis, and long multi-file sessions

Source: [Claude Code Model Configuration](#) — Official Anthropic Documentation